

Understanding AIS

part 4: scs interfaces

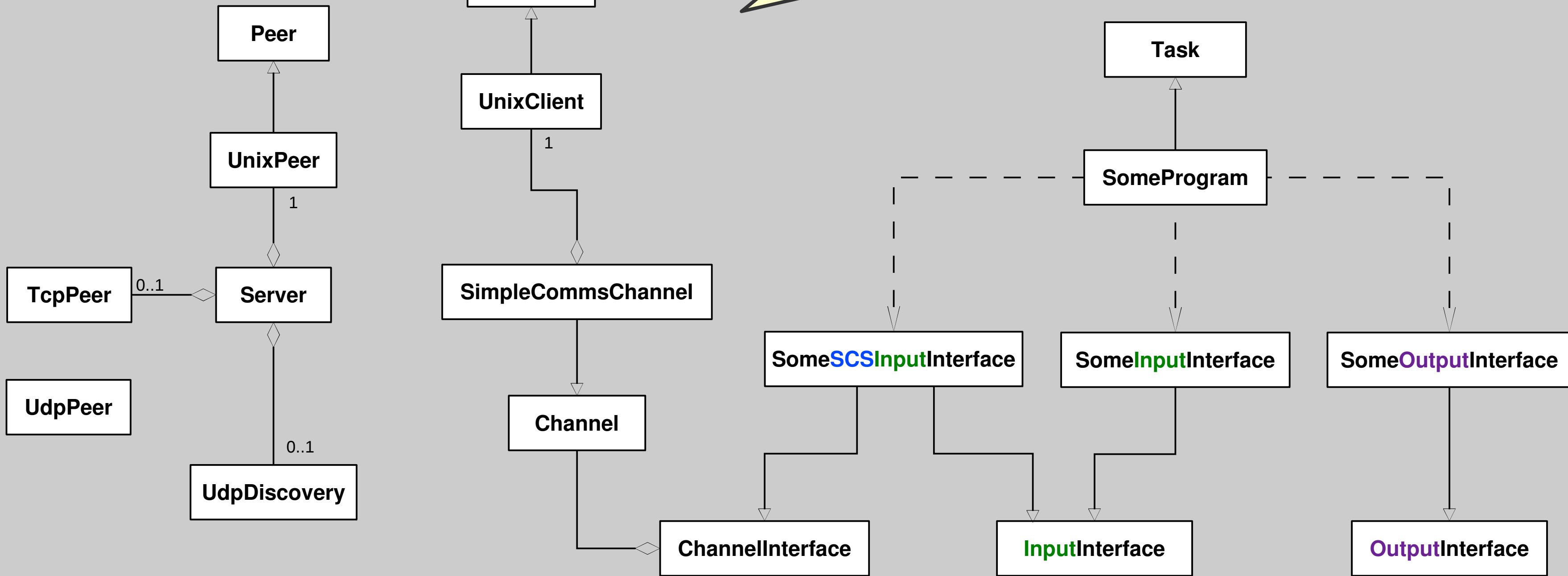
Introduction

- ◆ AIS stands for Autonomy Infrastructure Software
- ◆ AF stands for Application Framework
- ◆ SCS stands for Simple Communication System
- ◆ Part 1 covers SCS
- ◆ Part 2 covers Autonomy Tasks
- ◆ Part 3 covers Input & Output interfaces in general
- ◆ Part 4 covers SCS interfaces

SCS + AF

ECU #1

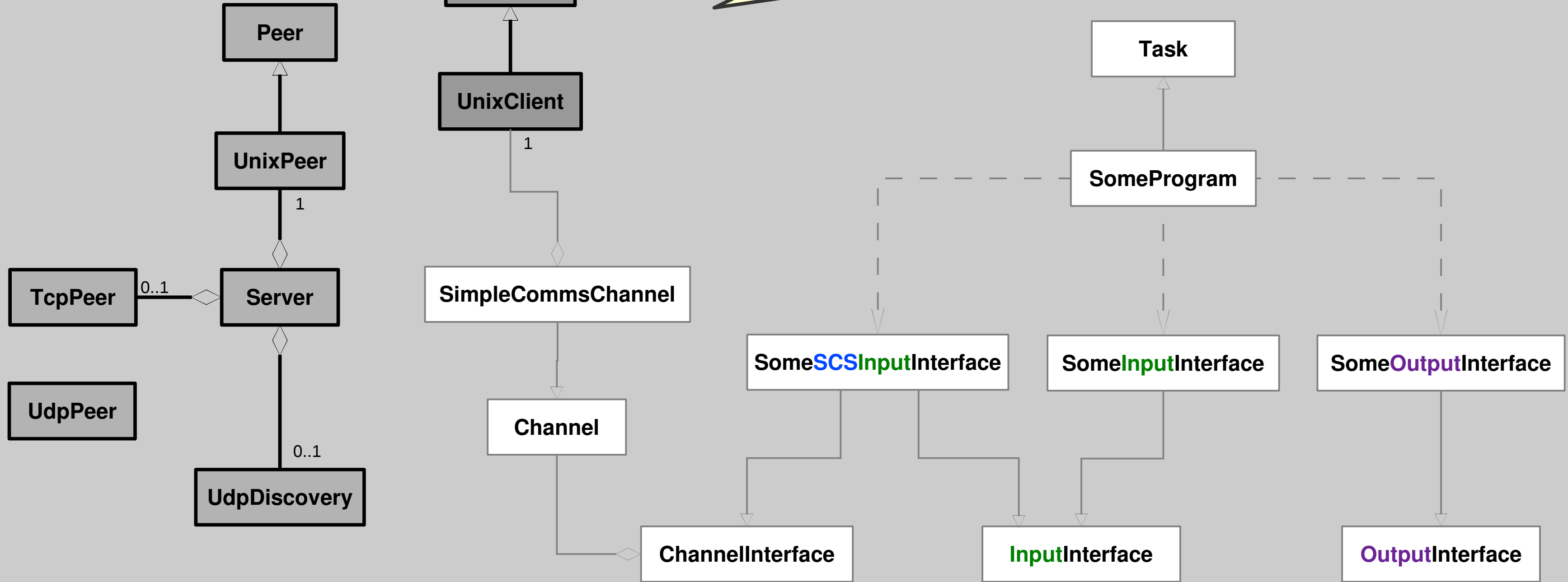
The high level view



scs server

ECU #1

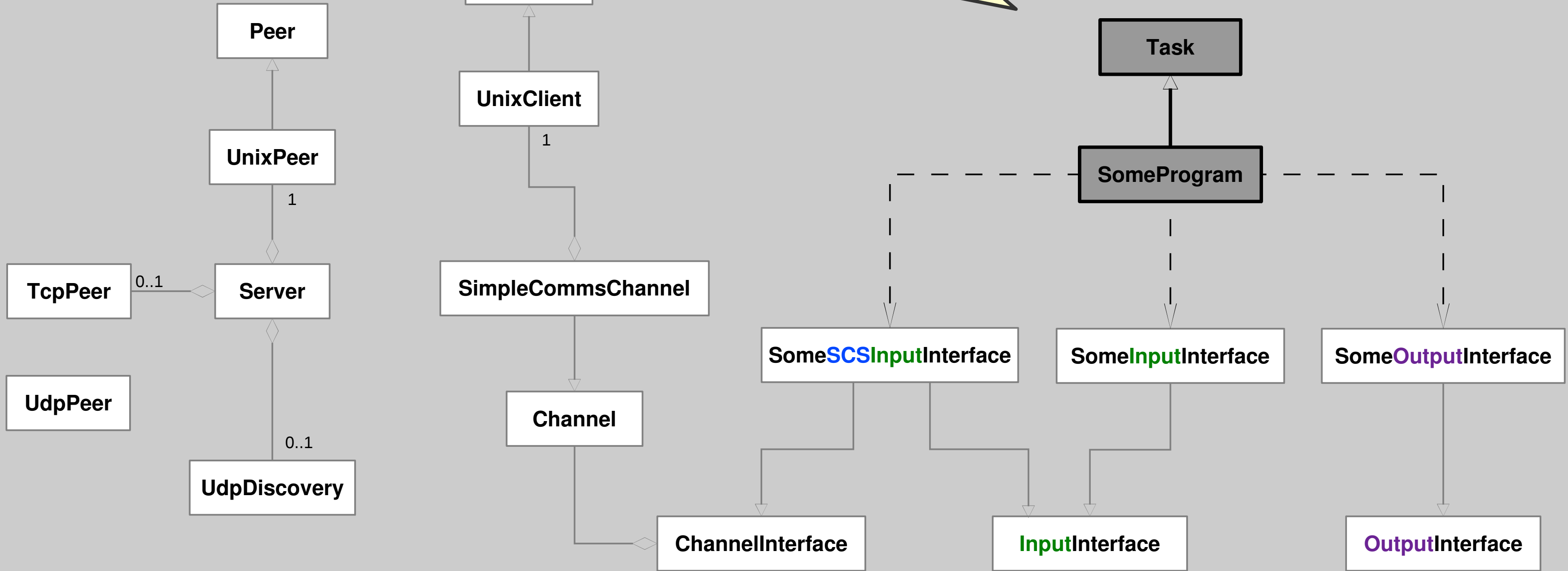
See AIS training part 1



Autonomy Task

ECU #1

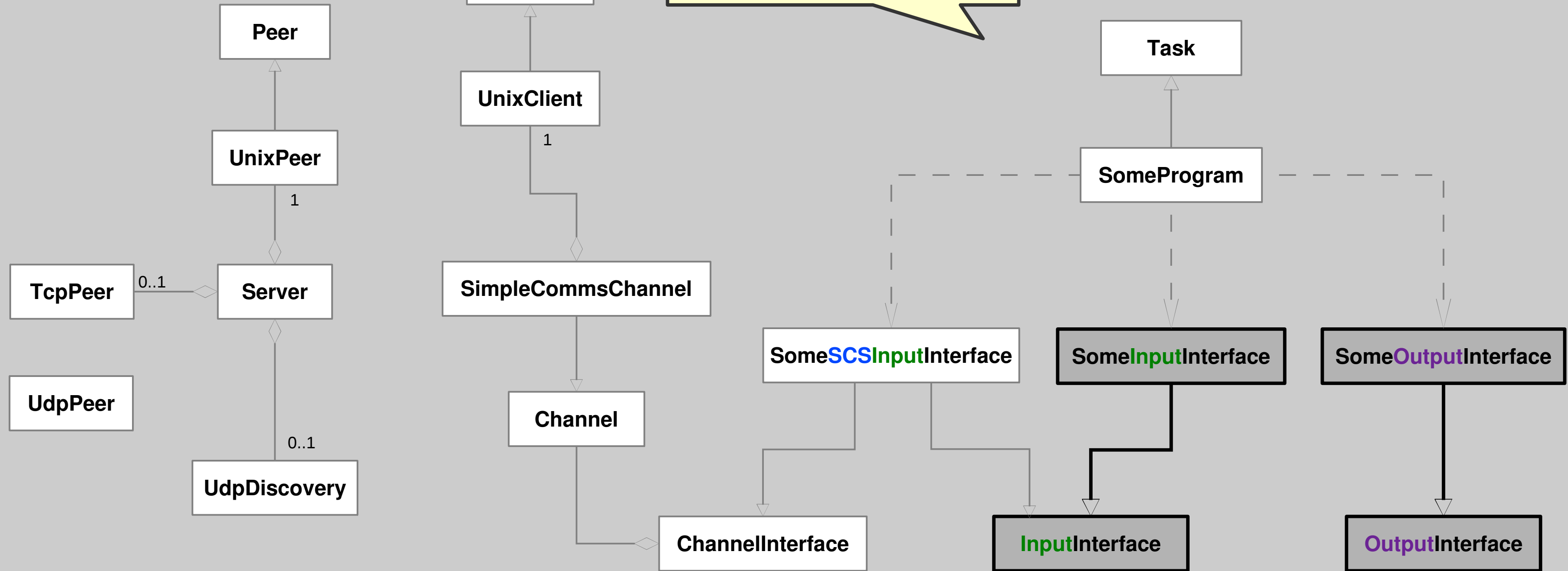
See AIS training part 2



this document scope

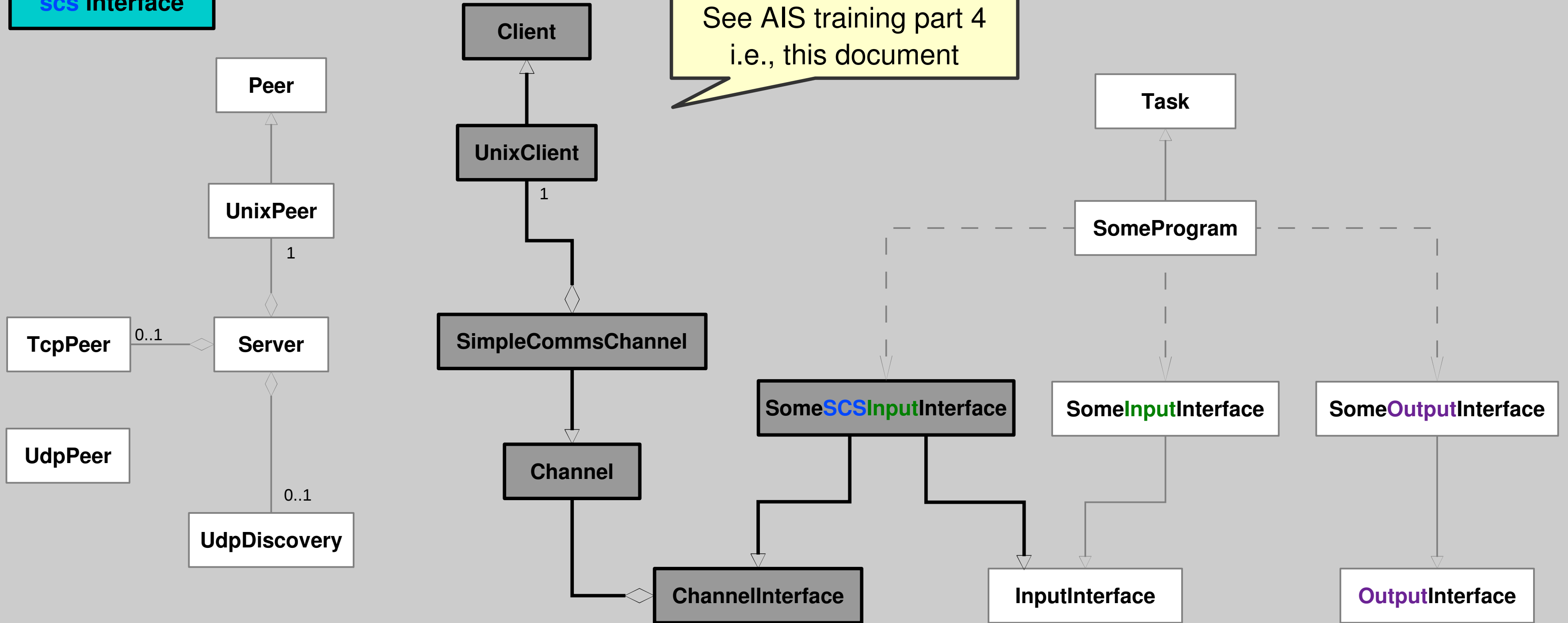
ECU #1

See AIS training part 3

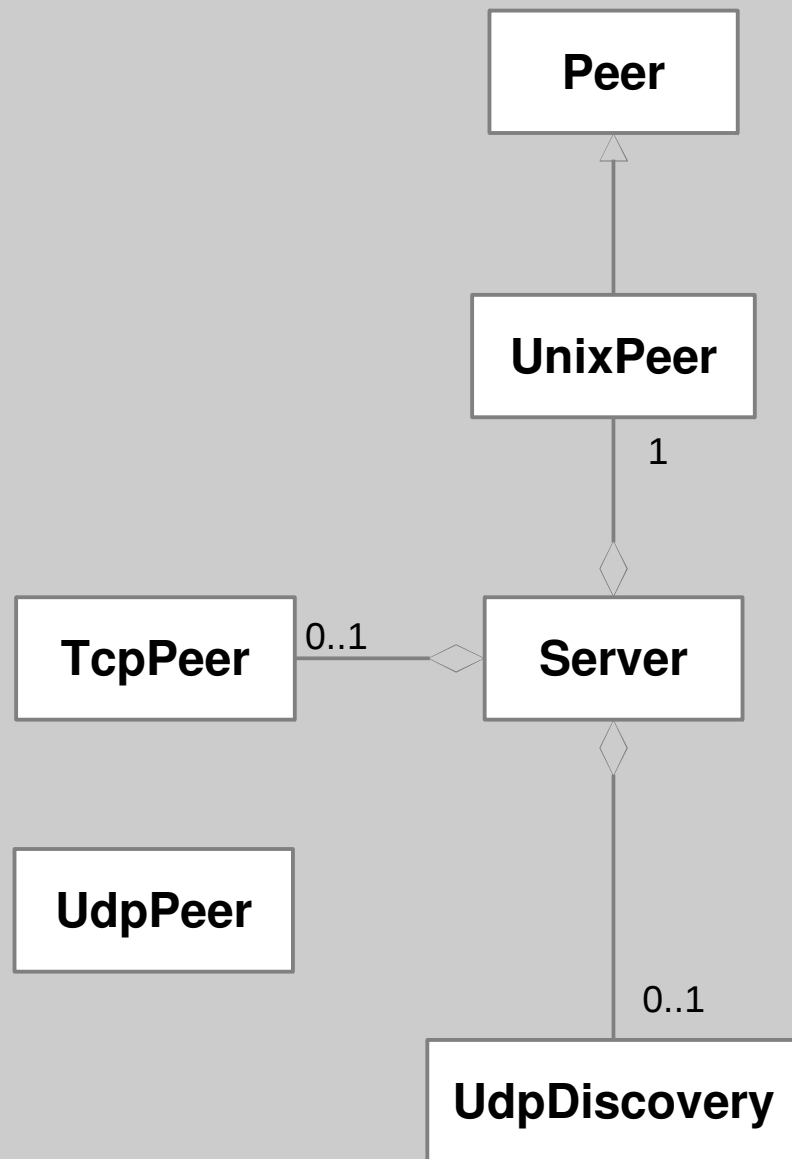


scs interface

ECU #1



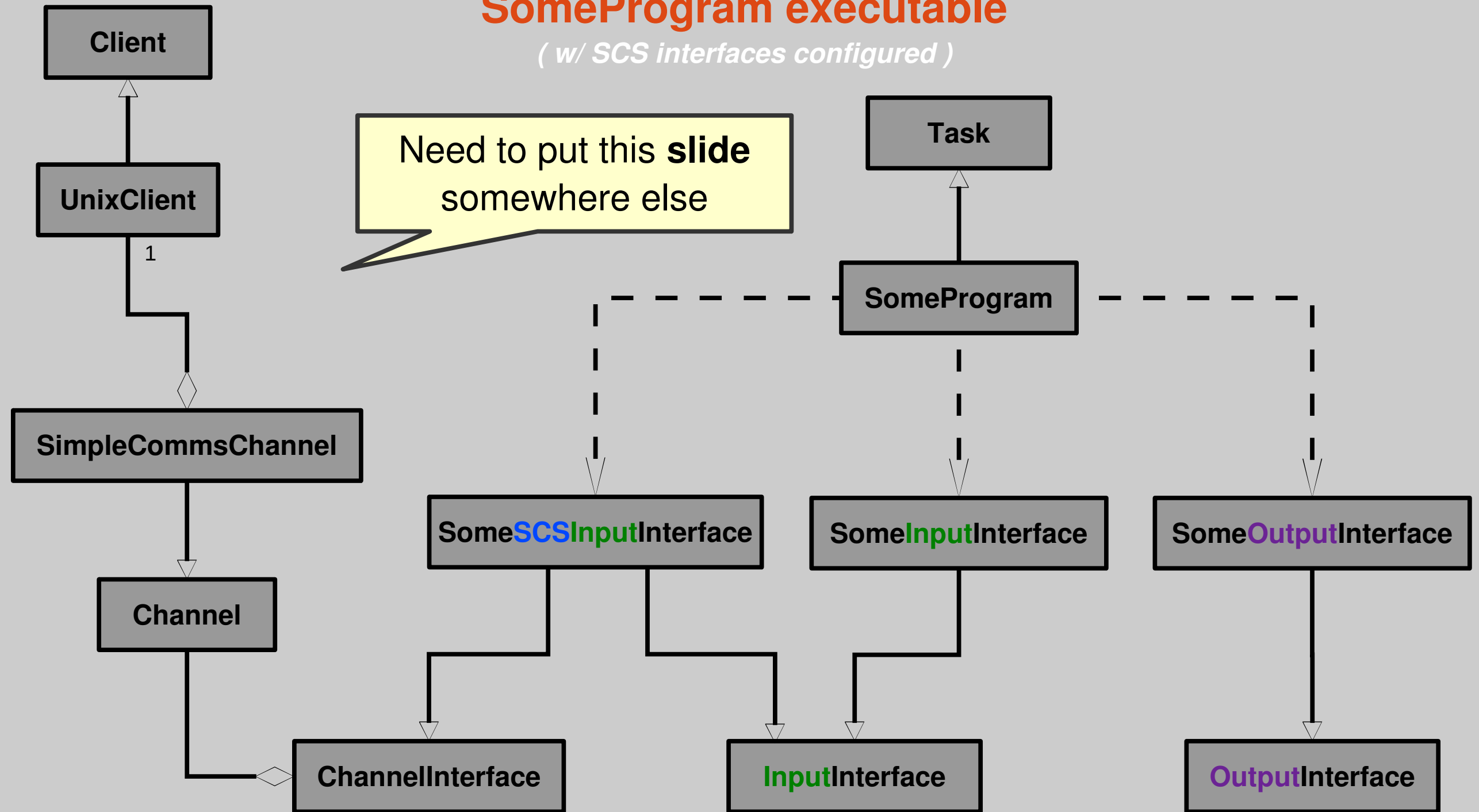
**Autonomy Task
w/**scs** interface(s)**



SomeProgram executable

(w/ SCS interfaces configured)

Need to put this **slide**
somewhere else



definition

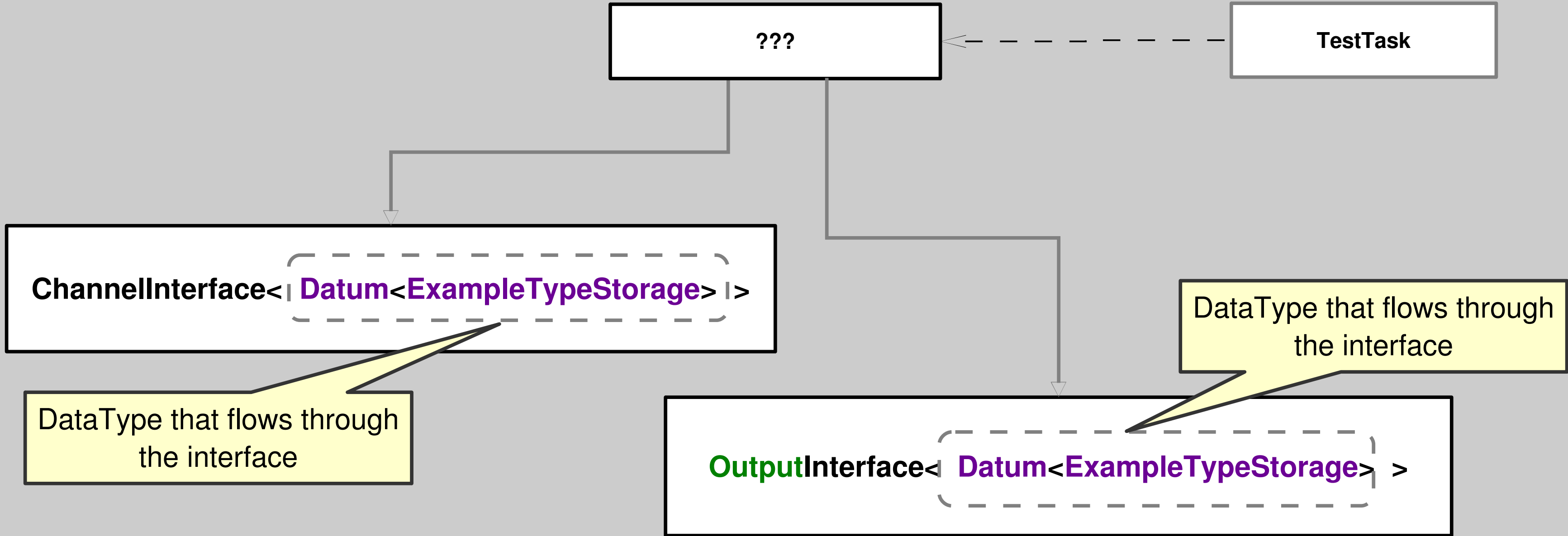
Question:

- ◆ What is a scs interface ?

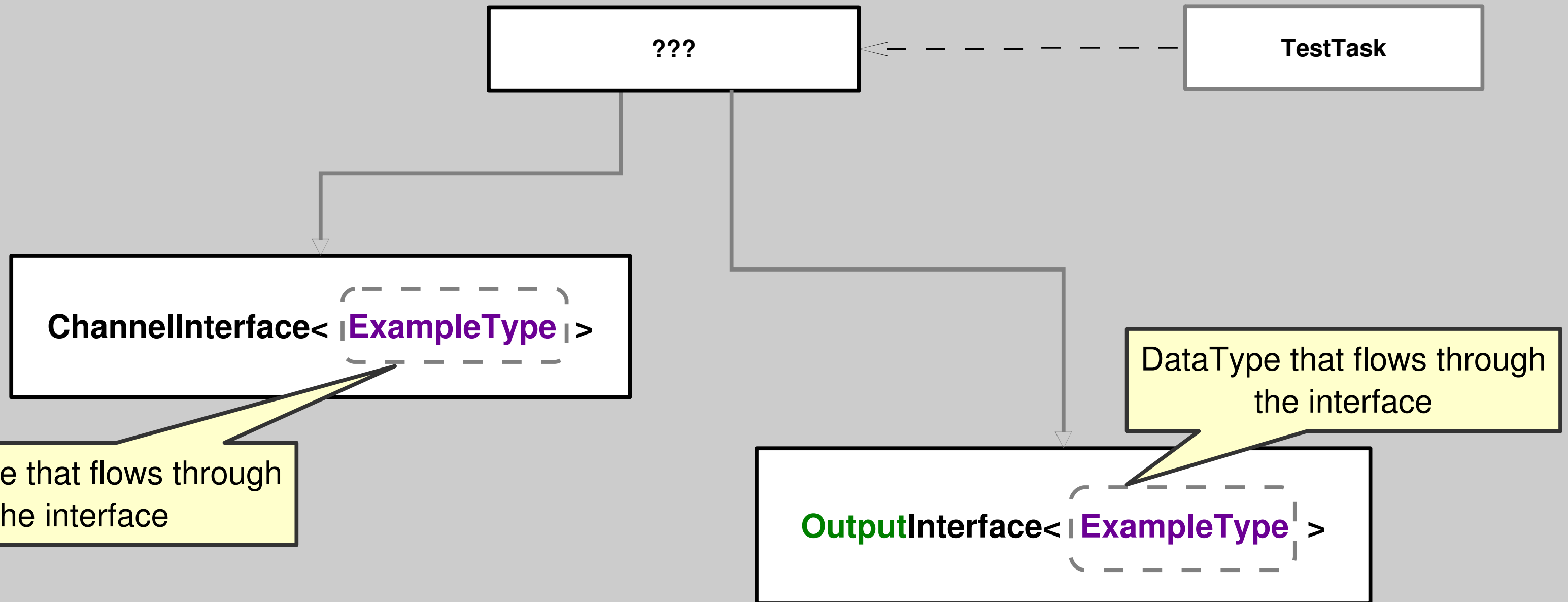
Answer:

- ◆ scs interface is either an Input or Output **Interface implementation**.
- ◆ The data that **flows through** this interface is transported using **scs**.
- ◆ If it is an input scs interface, then the data read from this interface is delivered by scs from a producer
- ◆ If it is an output scs interface, then the data published over this interface is sent to potential consumers using scs

Example scs output interface



Example - compact notation



implementation - naming

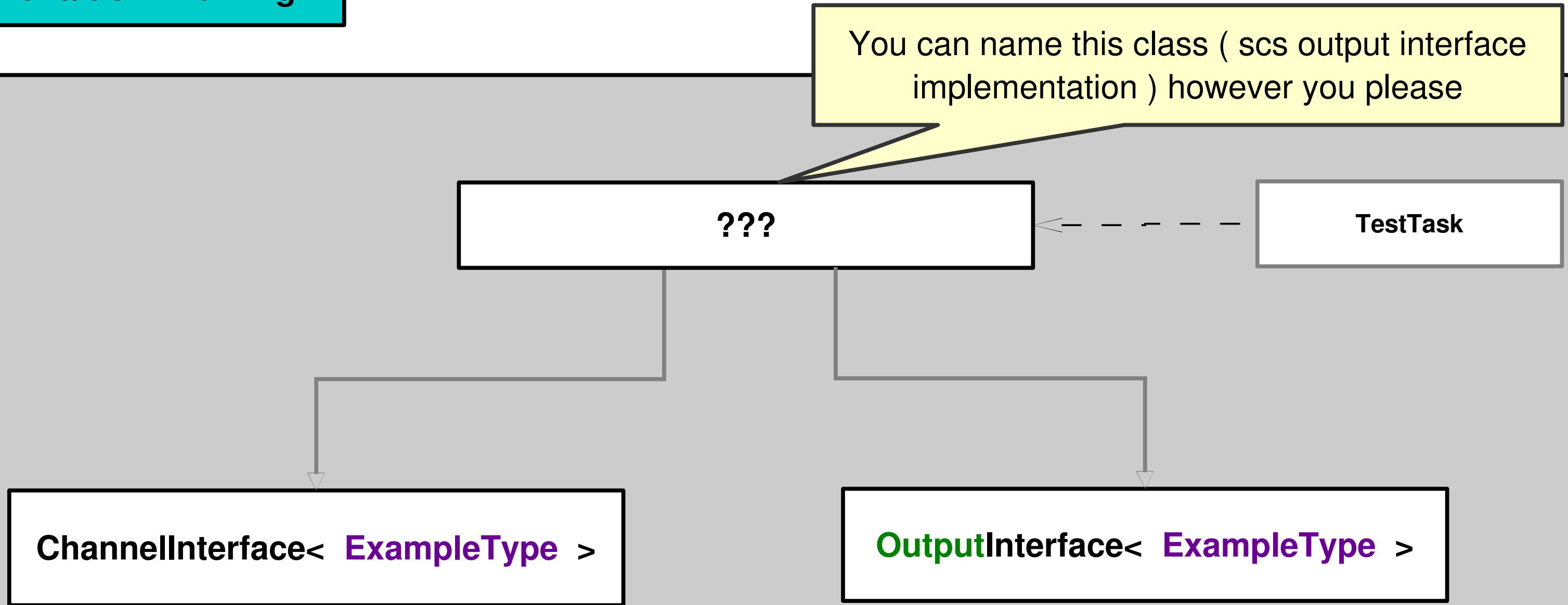
You can name this class (scs output interface implementation) however you please

???

TestTask

ChannelInterface< ExampleType >

OutputInterface< ExampleType >



Example - AIS naming standard

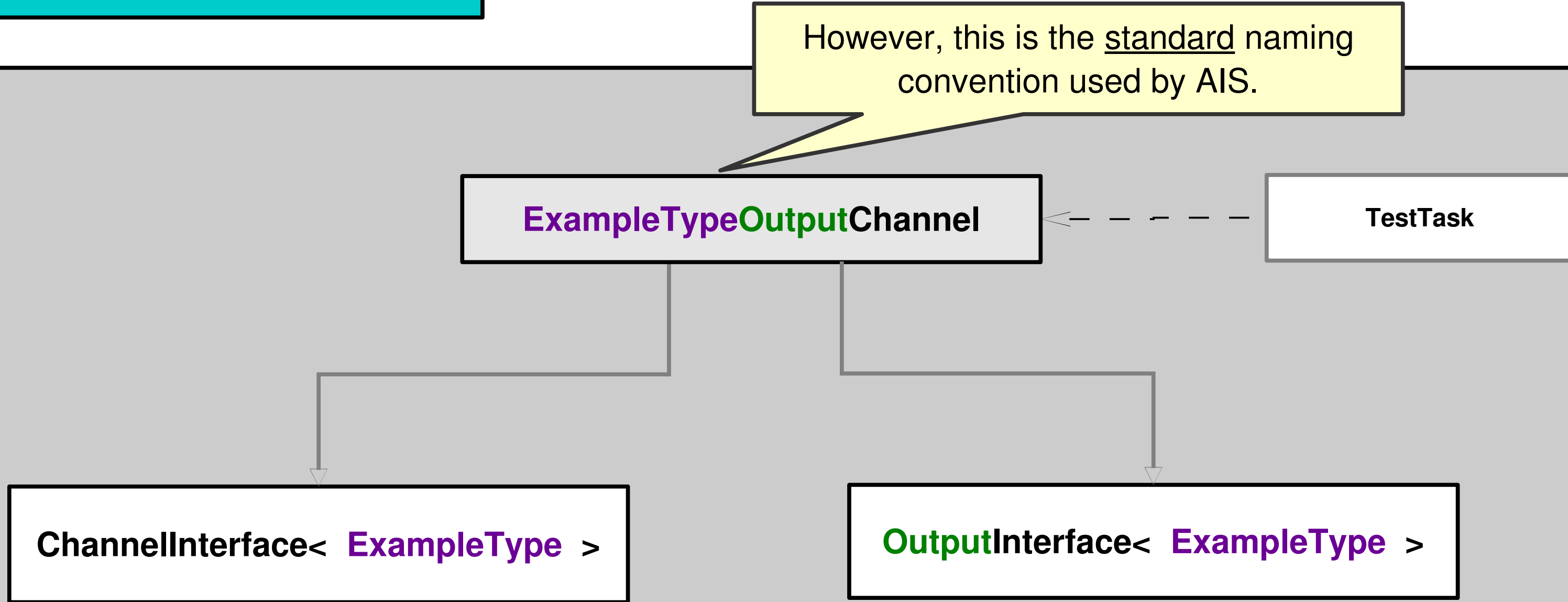
However, this is the standard naming convention used by AIS.

ExampleType**OutputChannel**

TestTask

ChannelInterface< **ExampleType** >

OutputInterface< **ExampleType** >



C++ class declaration

See next page how to declare
ExampleType**OutputChannel** in C++ ?

ExampleType**OutputChannel**

ChannellInterface< **ExampleType** >

OutputInterface< **ExampleType** >

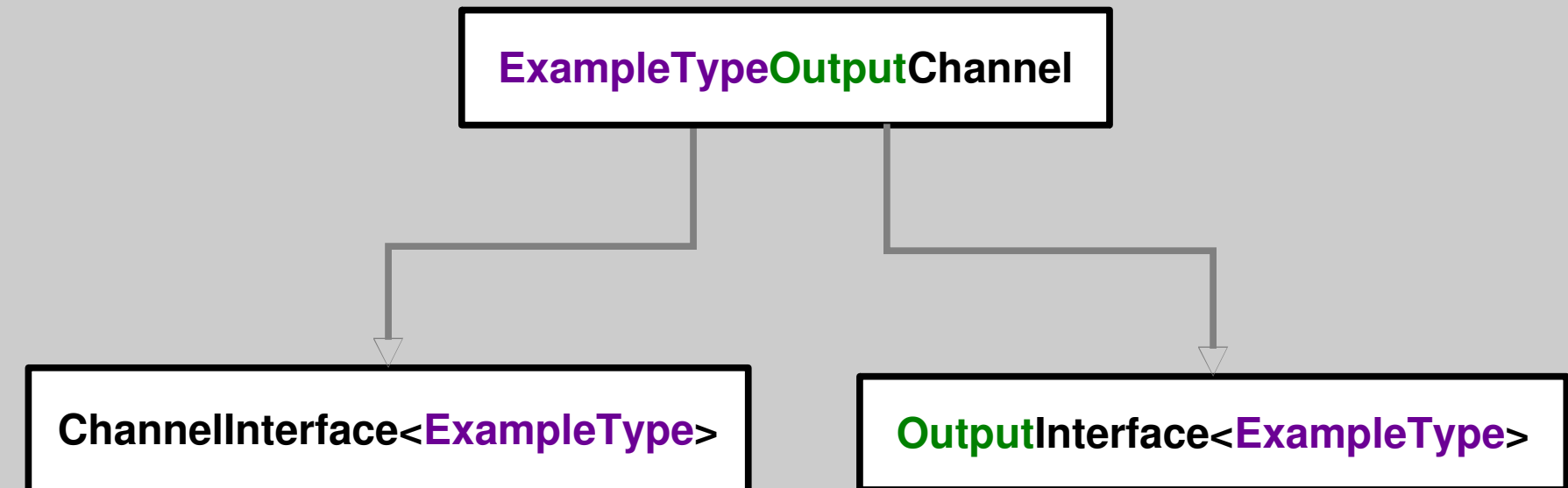


class declaration - cont'd

This is an example of multiple inheritance.

```
class ExampleTypeOutputChannel:    public ChannelInterface<ExampleType>, public OutputInterface<ExampleType>
{
    insert here class methods & data memembers
};
```

The derived class inherits from two base classes



Conveniency macro

ExampleType**OutputChannel**

TestTask

ALS provides a convenient macro to automate
declaration of this derived class.

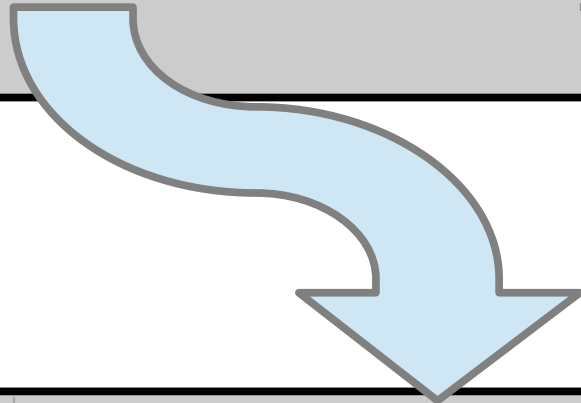


Conveniency macro syntax

enter anything - not used

The type that flows through the interface

Define**OutputChannel**(Example.Channel.Output, **ExampleType**)



```
class ExampleTypeOutputChannel: public ChannelInterface<ExampleTypeExampleType>
{
    ... and appropriate class methods & data memebbers are automatically inserted
};
```

Expansion

Define**OutputChannel**(Example.Channel.Output, **ExampleType**)

```
#define DefineOutputChannel(IntfAddress, Type) \  
class Type ## OutputChannel :    public task::ChannelInterface<Type>,\  
                                public OutputInterface<Type>  
{  
    public:  
        Type ## OutputChannel(ConfigSection &cs):        ChannelInterface<Type>(cs, Write),  
                                                         OutputInterface<Type>(cs) { };  
  
        virtual bool publish( const Type &val ) {  
            return (ChannelInterface<Type>::write(val) == task::CS_Success);  
        }  
};
```

prod/common/interfaces/baseTypes/
autogen_channel.h

Substitution

Define**OutputChannel**(Example.Channel.Output, **ExampleType**)

```
#define DefineOutputChannel(IntfAddress, ExampleType)  
class ExampleTypeOutputChannel :      public ChannelInterface<ExampleType>,  
                                     public OutputInterface<ExampleType>  
{  
    public:  
        ExampleTypeOutputChannel(ConfigSection &cs):  ChannelInterface<ExampleType>(cs, Write),  
                                                         OutputInterface<ExampleType>(cs) { };  
  
    virtual bool publish( const ExampleType &val ) {  
        return ( ChannelInterface<ExampleType>::write(val) == CS_Success );  
    }  
};
```

prod/common/interfaces/baseTypes/
autogen_channel.h

Implementation

DefineOutputChannel(Example.Channel.Output, **ExampleType**)

```
#define DefineOutputChannel(IntfAddress, ExampleType)  
class ExampleTypeOutputChannel :      public ChannelInterface<ExampleType>,  
                                     public OutputInterface<ExampleType>  
{  
    public:  
        ExampleTypeOutputChannel(ConfigSection &cs): ChannelInterface<ExampleType>(cs, Write),  
                                                    OutputInterface<ExampleType>(cs) { };  
  
    virtual bool publish( const ExampleType &val ) {  
        return ( ChannelInterface<ExampleType>::write(val) == CS_OK );  
    }  
};
```

DefineOutputChannel provides both declaration and implementation for **ExampleTypeOutputChannel**.
No .cpp file is needed for the class.

channel.cpp

```
#include "channel.h"  
PluginInterfaceDbRegistration(  
    Example.Channel.Output,  
    ExampleTypeOutputChannel )
```

```
iface = baseEnv.SharedLibrary(  
    target='Example.Channel.Output.so',  
    source='channel.cpp',  
    LIBS = [  
        baseEnv['BOOST_SERIALIZATION']  
    ]  
)  
  
baseEnv.InstallLibrary( iface )  
baseEnv.Alias("ExampleInterface", "#lib/libExample.Channel.Output.so")
```

Excerpt from interfaces.rb

```
ExampleOutput = {  
    "interface" => "Example.Channel.Output",  
  
    "channel" => {  
        "type"           => "SimpleComms",  
        "queueSize"      => 1,  
        "name"           => "Example.Channel",
```

